ONGConecta — Plataforma Web e Mobile

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.0

Campo	Descrição
Projeto	ONGConecta — Plataforma Web e Mobile para Gestão de Voluntários e Doações (nome ilustrativo)
Gerente de Projeto / Product Owner	Desenvolvedor Líder (acumula a função, dado o porte reduzido da equipe)
Gerente/Responsável de GCS-DevOps	Desenvolvedor Líder
Fábrica de Software / Squad	Squad voluntária remota — 1 Desenvolvedor Líder + 3 Desenvolvedores Júnior
Repositório principal	https://github.com/lzcamargo/ongconecta (GitHub, plano gratuito)
Regime de trabalho	100% remoto, voluntário, dedicação parcial e assíncrona

Histórico de Revisões

Data	Versão	Descrição	Autor
24/08/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD), adaptado para equipe voluntária remota de 4 pessoas desenvolvendo aplicação web e mobile para a ONG.	Desenvolvedor Líder

Sumário

Histórico de Revisões 2

1. Introdução 4

1.1 Finalidade 4

1.2 Escopo 4

1.3 Definições, Acrônimos e Abreviações 4

1.4 Referências 5

1.5 Visão Geral 5

2. Cultura DevOps e Papéis na Gerência de Configuração 5

2.1 Comunicação assíncrona 6

3. Ferramentas, Ambientes e Infraestrutura 6

3.1 Ferramentas do Pipeline 7

3.2 Ambientes e Infraestrutura 8

3.3 Estratégia de Branching e Versionamento 8

4. Identificação da Configuração 8

4.1 Itens de Configuração 8

4.2 Convenção de Nomenclatura 9

4.3 Estrutura do Repositório 9

5. Integração Contínua (CI) 9

5.1 Etapas do Pipeline de CI 9

5.2 Política de Pull Request e Revisão de Código 10

5.3 Critérios de Qualidade (Quality Gate) 10

6. Entrega e Implantação Contínua (CD) 10
6.1 Etapas do Pipeline de CD — Web 10
6.2 Etapas do Pipeline de CD — Mobile 10
6.3 Estratégias de Implantação 10
6.4 Rollback e Recuperação 11
7. Infraestrutura como Código (IaC) 11
8. Controle de Configuração e Mudança 11
8.1 Processo de Solicitação de Mudança 11
8.2 Comitê de Controle de Mudança (CCM) 11
9. Baselines e Releases 12
9.1 Geração Automática de Baseline 12
9.2 Notas de Versão (Release Notes) 12
10. Estimativa do Status de Configuração 12
10.1 Monitoramento e Observabilidade 12
10.2 Métricas acompanhadas 12
10.3 Relatórios e Auditorias 12
11. Marcos 12
12. Treinamento e Recursos 13
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros 13
14. Quadro-Resumo do Pipeline CI/CD 13

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida da aplicação web e do aplicativo mobile desenvolvidos para a ONG, com ênfase em práticas DevOps — Integração Contínua (CI), Entrega/Implantação Contínua (CD), automação de pipelines e uso de ferramentas gratuitas ou de baixo custo, compatíveis com a realidade de um projeto sem fins lucrativos mantido por equipe voluntária.

Diferentemente do template genérico de referência, este plano parte de três restrições concretas do projeto: (1) equipe pequena — 1 desenvolvedor líder e 3 desenvolvedores júnior; (2) regime 100% voluntário e remoto, com disponibilidade assíncrona e variável entre os membros; e (3) ausência de orçamento para infraestrutura própria. Essas restrições levaram a simplificações deliberadas em relação ao template original (por exemplo, dispensa de Kubernetes/Terraform em favor de plataformas gerenciadas — PaaS —, e consolidação de papéis que no template estão descritos separadamente).

1.1 Finalidade

Estabelecer um padrão simples e sustentável a ser seguido pelos quatro integrantes da squad, garantindo rastreabilidade, versionamento e controle de mudanças da aplicação web e do aplicativo mobile, apoiado por pipelines automatizados de build, teste, integração e implantação — com o mínimo de burocracia possível, já que a equipe não dispõe de tempo dedicado em período integral.

1.2 Escopo

Este PGC é destinado a todos os integrantes da squad voluntária responsável pelo desenvolvimento e operação da plataforma da ONG, composta por:

- 1 Desenvolvedor Líder — atua também como Gerente de Projeto, Engenheiro DevOps/Release Engineer e responsável final por decisões de arquitetura e segurança;
- 3 Desenvolvedores Júnior — responsáveis pela implementação de funcionalidades, testes automatizados e revisão cruzada de código, em regime remoto e assíncrono.

O escopo abrange dois entregáveis versionados de forma coordenada: (a) a aplicação web (frontend + backend/API) e (b) o aplicativo mobile (Android e iOS), incluindo pipelines, scripts de build, configuração de ambientes e artefatos publicados nas lojas de aplicativos.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
PaaS	Plataforma como Serviço (Platform as a Service) — hospedagem gerenciada usada no lugar de infraestrutura própria
CCM	Comitê de Controle de Mudanças (aqui reduzido a Dev. Líder + 1 Dev. Júnior)

Termo	Significado
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
GitHub Flow	Estratégia de branching simplificada, com branch principal sempre implantável e branches de feature de vida curta
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH)
APK/AAB / IPA	Formatos de pacote de aplicativo Android (APK/AAB) e iOS (IPA)
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate) — aqui acompanhadas de forma simplificada

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.
- Documentação oficial do GitHub Actions, Google Play Console e App Store Connect.

Inserir demais referências específicas do projeto: Plano do Projeto, Cronograma de voluntariado, Documento de Arquitetura, Termo de adesão de voluntários, etc. Caso disponível

1.5 Visão Geral

Assim como no template de referência, a GCS é tratada aqui como parte constante do pipeline de entrega, e não como atividade manual concentrada em marcos formais: cada Pull Request revisado, testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados nas lojas). Isso é especialmente importante para uma equipe voluntária, pois reduz a dependência de conhecimento tácito e facilita a integração de novos voluntários caso haja rotatividade na equipe — risco real e esperado neste tipo de projeto.

2. Cultura DevOps e Papéis na Gerência de Configuração

Dado o porte da equipe (4 pessoas), os papéis do template original foram consolidados. Não há squads de SRE, Segurança ou QA dedicadas: essas responsabilidades são compartilhadas por todos, com o Desenvolvedor Líder atuando como ponto final de decisão em mudanças de maior risco.

Papel (template)	Quem exerce no projeto	Responsabilidades
Gerente de Projeto / Product Owner	Desenvolvedor Líder	Prioriza o backlog (GitHub Projects/Issues), decide o que entra em cada release e comunica prazos à ONG.

Papel (template)	Quem exerce no projeto	Responsabilidades
Engenheiro DevOps / Release Engineer	Desenvolvedor Líder	Mantém os pipelines de CI/CD, a estratégia de branching/versionamento e a publicação nas lojas de aplicativos.
Desenvolvedor(a)	3 Desenvolvedores Júnior + Desenvolvedor Líder	Implementam funcionalidades, escrevem testes automatizados e revisam Pull Requests dos colegas (revisão cruzada obrigatória).
SRE / Operações	Rotativo entre os 4 integrantes (escala semanal informal)	Acompanha alertas de monitoramento gratuito (ex.: UptimeRobot, Sentry) e comunica incidentes no canal do time.
QA / Automação de Testes	Desenvolvedores Júnior, com apoio do Líder	Mantêm a suíte de testes automatizados executada no pipeline; não há QA dedicado — testes automatizados substituem QA manual formal.
Segurança (DevSecOps)	Desenvolvedor Líder	Habilita e revisa alertas do Dependabot e do scanner de segredos do GitHub; define o que pode ser exposto em configuração pública.
Comitê de Controle de Mudanças (CCM)	Desenvolvedor Líder + 1 Desenvolvedor Júnior indicado	Atua apenas em mudanças de alto risco (ex.: mudança de esquema de banco em produção); mudanças de rotina seguem o fluxo automatizado de PR.

2.1 Comunicação assíncrona

Como a equipe é 100% remota e voluntária, com disponibilidade variável, o plano assume comunicação predominantemente assíncrona. Recomenda-se:

- Descrições de Pull Request completas (contexto, decisão tomada, como testar), para que a revisão não dependa de reunião síncrona;
- Uso de um canal único (ex.: Discord/Slack gratuito) para avisos de deploy, incidentes e dúvidas de revisão;
- Janela combinada de tempo máximo de resposta a um Pull Request (ex.: até 3 dias úteis), para não travar o fluxo de entrega.

3. Ferramentas, Ambientes e Infraestrutura

A seleção de ferramentas priorizou soluções gratuitas ou com planos gratuitos generosos para projetos pequenos/open source, evitando custos de infraestrutura para a ONG. Sempre que possível, plataformas gerenciadas (PaaS) substituem infraestrutura própria (servidores, Kubernetes), reduzindo a carga operacional sobre uma equipe voluntária de 4 pessoas.

3.1 Ferramentas do Pipeline

Categoria	Ferramenta(s) adotada(s)	Finalidade
Controle de versão	Git + GitHub (plano gratuito / GitHub for Nonprofits)	Versionamento de código-fonte, configuração de pipelines como código e documentação
Orquestração de CI/CD	GitHub Actions (minutos gratuitos do plano)	Automação de build, testes, análise e implantação para web e mobile
Hospedagem Web (PaaS)	Vercel / Netlify (frontend) + Render / Railway (backend/API) — planos gratuitos	Substitui infraestrutura própria; deploy automático a partir do repositório, sem necessidade de gerenciar servidores
Build e distribuição Mobile	EAS Build (Expo) ou Fastlane, conforme stack escolhida (React Native/Flutter nativo)	Geração automatizada dos pacotes Android (AAB) e iOS (IPA)
Publicação Mobile	Google Play Console + Apple App Store Connect (contas da ONG)	Distribuição oficial do aplicativo mobile e canais de teste (Internal Testing / TestFlight)
Gestão de segredos	GitHub Secrets / Environments	Armazenamento das credenciais usadas nos pipelines, sem custo adicional de cofre dedicado
Monitoramento e Observabilidade	Sentry (free tier) + UptimeRobot (free)	Rastreamento de erros em produção e verificação de disponibilidade (uptime) do backend
Gestão de dependências / SCA	Dependabot (nativo do GitHub, gratuito)	Identificação de vulnerabilidades em bibliotecas de terceiros, com PRs automáticos de atualização
Rastreamento de tarefas	GitHub Issues + GitHub Projects	Vinculação de commits/PRs a itens de trabalho, sem necessidade de ferramenta paga separada

3.2 Ambientes e Infraestrutura

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento / Preview	Ambiente efêmero criado automaticamente por Pull Request (Preview Deploy da Vercel/Netlify/Render), permitindo revisar cada mudança antes do merge.	Abertura ou atualização de Pull Request
Homologação / Staging	Validação funcional da web e canais de teste do app mobile (Internal Testing no Google Play; TestFlight na Apple).	Merge na branch principal (main)
Produção	Ambiente utilizado pela ONG e pelo público final (site/app publicado nas lojas).	Aprovação manual do Desenvolvedor Líder após validação em staging

3.3 Estratégia de Branching e Versionamento

Recomenda-se o uso de GitHub Flow em vez do Trunk-Based Development puro sugerido no template original, por ser mais simples de seguir por desenvolvedores júnior e mais tolerante a branches de feature com vida um pouco mais longa, compatível com a disponibilidade assíncrona de voluntários:

- main — sempre implantável (deployable); protegida por regra de PR obrigatório com pipeline verde e ao menos 1 aprovação;
- feature/<descrição-curta> — uma branch por funcionalidade ou correção, aberta por qualquer um dos 4 integrantes;
- release/x.y (opcional) — usada apenas antes de uma submissão às lojas de aplicativos, para estabilizar a versão durante o período de revisão da Apple/Google;
- hotfix/<descrição> — correções emergenciais, sempre revisadas pelo Desenvolvedor Líder antes do merge.

Versionamento: adoção de Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits (feat, fix, chore, docs, refactor…), permitindo geração automática de changelog e da próxima versão pelo pipeline, tanto para a web quanto para o app mobile (versionCode/versionName no Android e CFBundleVersion no iOS derivados do mesmo número semântico).

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, os seguintes artefatos são itens de configuração formais, versionados no mesmo repositório (ou repositórios companheiros para web e mobile):

- Código-fonte da aplicação web (frontend e backend/API) e dos testes automatizados;
- Código-fonte do aplicativo mobile (Android/iOS ou multiplataforma) e seus testes;
- Definições de pipeline como código (.github/workflows);

- Arquivos de configuração de build mobile (app.json/app.config.js, build.gradle, Info.plist, ícones e splash screens);
- Arquivos de configuração por ambiente (variáveis, feature flags) — exceto segredos, que ficam no GitHub Secrets;
- Scripts de migração de banco de dados;
- Metadados de publicação nas lojas (descrição, screenshots, política de privacidade) — versionados em pasta própria do repositório;
- Documentação técnica (README, ADRs — Architecture Decision Records) e guia de onboarding para novos voluntários.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<issue>-<descrição-curta>	feature/42-cadastro-de-doacoes
Commit	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(mobile): adiciona tela de login
Tag de release	Versionamento Semântico vX.Y.Z	v1.2.0
Build mobile	<plataforma>-<versão>-<build>	android-1.2.0-45 / ios-1.2.0-45

4.3 Estrutura do Repositório

Sugestão de estrutura, a ser confirmada conforme a stack técnica escolhida pela equipe: /web (frontend), /api (backend), /mobile (aplicativo), /.github/workflows (pipelines), /docs (README, ADRs, guia de onboarding de voluntários), /tests. Caso web e mobile compartilhem lógica (ex.: monorepo com React/React Native), documentar a decisão em um ADR.

5. Integração Contínua (CI)

Todo Pull Request para a branch main deve disparar automaticamente um pipeline de CI, tanto para o código web quanto para o mobile (pipelines separados, disparados condicionalmente conforme a pasta alterada). Nenhuma mudança é integrada sem que o pipeline seja concluído com sucesso.

5.1 Etapas do Pipeline de CI

- Checkout do código e restauração de dependências (com cache, para reduzir consumo dos minutos gratuitos do GitHub Actions);
- Build da aplicação web e/ou do aplicativo mobile, conforme os arquivos alterados;
- Análise estática de código (lint);
- Testes unitários, com meta de cobertura mínima adequada ao porte da equipe (ver Quality Gate);
- Testes de integração, quando aplicável;
- Análise de dependências (Dependabot/SCA);
- Publicação do artefato de build (web) ou do pacote de teste interno (mobile) para revisão.

5.2 Política de Pull Request e Revisão de Código

- Todo PR exige, no mínimo, 1 aprovação de outro integrante da equipe antes do merge — recomenda-se que PRs de desenvolvedores júnior sejam revisados pelo Desenvolvedor Líder sempre que possível, dado o menor tempo de experiência da equipe;
- PRs devem ser pequenos e descritivos, para viabilizar revisão assíncrona sem necessidade de reunião;
- Merge somente permitido com pipeline de CI verde (quality gate).

5.3 Critérios de Qualidade (Quality Gate)

[Definir limites objetivos e realistas para o porte da equipe, ex.: cobertura de testes ≥ 60% nas áreas críticas (autenticação, doações/pagamentos), zero vulnerabilidades críticas apontadas pelo Dependabot, zero erros de lint bloqueante. Metas mais altas podem ser adotadas progressivamente conforme a maturidade da suíte de testes evoluir.]

6. Entrega e Implantação Contínua (CD)

Após a publicação do artefato pelo pipeline de CI, o pipeline de CD promove essa mesma versão até staging e, mediante aprovação, até produção — com fluxos distintos para web e para mobile, já que a publicação em lojas de aplicativos envolve revisão externa (Apple/Google), fora do controle da equipe.

6.1 Etapas do Pipeline de CD — Web

- Deploy automático em ambiente de Homologação/Staging (PaaS);
- Execução de testes automatizados de aceitação em staging, quando existentes;
- Aprovação manual do Desenvolvedor Líder para produção;
- Deploy em produção via PaaS (rolling update nativo da plataforma);
- Verificação de saúde (health check) pós-deploy;
- Geração automática de tag de release e notas de versão (changelog).

6.2 Etapas do Pipeline de CD — Mobile

- Build automatizado do pacote (AAB/IPA) a cada merge na main;
- Publicação automática no canal de teste interno (Internal Testing / TestFlight) para validação pela equipe;
- Aprovação manual do Desenvolvedor Líder para submissão às lojas;
- Submissão à revisão da Google Play e da Apple App Store (prazo fora do controle da equipe — normalmente de algumas horas a poucos dias);
- Lançamento faseado (staged rollout) quando suportado pela loja, para reduzir risco de uma versão com falhas críticas;
- Geração automática de tag de release correspondente à versão publicada.

6.3 Estratégias de Implantação

Estratégia	Uso no projeto
Rolling update (Web)	Padrão das plataformas PaaS escolhidas; não requer configuração manual adicional.
Lançamento faseado (Mobile)	Usado no Google Play para expor a nova versão gradualmente a uma fração dos usuários antes de 100%.

Estratégia	Uso no projeto
TestFlight / Internal Testing (Mobile)	Usado como "staging" do app mobile antes da submissão oficial às lojas.
Feature Flags	Recomendado para funcionalidades maiores (ex.: novo fluxo de doação), permitindo ativar/desativar sem novo deploy — mesmo com equipe pequena, evita reverter todo o app em caso de problema pontual.

6.4 Rollback e Recuperação

Web: a plataforma PaaS mantém builds anteriores disponíveis, permitindo rollback com um clique/comando, executado pelo Desenvolvedor Líder ou por qualquer integrante treinado para isso. Mobile: como a reversão de uma versão já publicada nas lojas não é imediata (nova revisão é necessária), recomenda-se uso de feature flags para desativar remotamente uma funcionalidade problemática enquanto uma correção é preparada, em vez de depender apenas de rollback de pacote.

7. Infraestrutura como Código (IaC)

Dado que o projeto utiliza plataformas gerenciadas (PaaS) em vez de infraestrutura própria, o uso de ferramentas completas de IaC (Terraform/Ansible) não é obrigatório neste momento. Ainda assim, o princípio de "tudo versionado" é mantido: a configuração de cada serviço (ex.: vercel.json, render.yaml, arquivos de configuração de build) deve ser versionada no repositório e revisada via Pull Request, e não alterada manualmente pelos painéis das plataformas.

- Alterações de configuração de plataforma seguem o mesmo processo de PR + revisão + pipeline usado para código de aplicação;
- Caso o projeto cresça e passe a exigir infraestrutura própria (ex.: servidores dedicados, banco de dados gerenciado fora do PaaS), este plano deve ser atualizado para incorporar Terraform ou ferramenta equivalente, retomando a seção correspondente do template original.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina (funcionalidades, correções) seguem o fluxo padrão: abertura de Issue no GitHub → branch de feature → Pull Request → revisão por outro integrante → pipeline de CI → merge → pipeline de CD. Esse fluxo, por si, já constitui o registro formal e rastreável da mudança, sem necessidade de formulários adicionais.

8.2 Comitê de Controle de Mudança (CCM)

Dada a equipe reduzida, o CCM é composto apenas pelo Desenvolvedor Líder e por 1 Desenvolvedor Júnior indicado (em rodízio, para desenvolver essa responsabilidade em mais de uma pessoa). É acionado apenas em mudanças de alto impacto ou risco elevado, como:

- Alterações de esquema de banco de dados em produção;
- Mudanças que envolvam dados sensíveis de doadores/voluntários da ONG (LGPD);
- Exceções ao fluxo padrão (ex.: hotfix emergencial publicado direto em produção).

Em situações rotineiras, a aprovação por pares (revisão de PR) substitui o comitê formal.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Cada implantação bem-sucedida em produção — tanto da web quanto do mobile — gera automaticamente uma baseline: uma tag de versão (SemVer) associada ao commit e, no caso mobile, ao build submetido às lojas.

9.2 Notas de Versão (Release Notes)

As notas de release são geradas automaticamente pelo pipeline a partir dos Conventional Commits desde a última tag, publicadas junto à release no repositório e reaproveitadas (de forma resumida) como texto de "novidades da versão" nas lojas de aplicativos.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

Os ambientes de Homologação e Produção devem ser monitorados por ferramentas gratuitas (Sentry para erros de aplicação, UptimeRobot para disponibilidade), com alertas encaminhados ao canal de comunicação da equipe. O responsável pelo monitoramento na semana (ver rodízio de SRE, item 2) acompanha esses alertas.

10.2 Métricas acompanhadas

Dada a escala do projeto, recomenda-se acompanhar de forma simplificada apenas duas das quatro métricas DORA do template original, para não gerar overhead desnecessário à equipe voluntária:

Métrica	Descrição	Por que priorizar
Deployment Frequency	Frequência com que o código é implantado em produção.	Indica se o time consegue entregar de forma contínua mesmo com disponibilidade parcial.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.	Sinal de qualidade mais crítico para um time júnior em formação; orienta reforço de testes.

Lead Time for Changes e MTTR podem ser adotadas posteriormente, quando o time tiver mais histórico de dados e maturidade de processo.

10.3 Relatórios e Auditorias

O histórico de pipelines (builds, testes, deploys) do GitHub Actions e os logs do sistema de versionamento constituem o principal insumo de auditoria de GCS, dispensando relatórios manuais.

11. Marcos

- Configuração do repositório e das regras de proteção da branch main;
- Implantação do pipeline de CI (web e mobile);

- Implantação do pipeline de CD para staging (web e canais de teste mobile);
- Primeiro deploy automatizado em produção (web);
- Primeira publicação do aplicativo nas lojas (Google Play e App Store);
- Primeira release totalmente automatizada, do commit à produção, sem intervenção manual além da aprovação de gate.

12. Treinamento e Recursos

Por se tratar de equipe voluntária com integrantes júnior, o treinamento é um insumo crítico de continuidade — mais até do que em um time profissional estável. **Recomenda-se:**

- Guia de onboarding versionado no repositório (/docs), cobrindo: como clonar o projeto, rodar localmente, abrir um PR e interpretar o resultado do pipeline;
- ADRs (Architecture Decision Records) curtos para decisões técnicas relevantes, evitando que o conhecimento fique apenas na cabeça do Desenvolvedor Líder;
- Revisões de PR usadas como oportunidade de mentoria (comentários explicativos, não apenas aprovação/rejeição), já que os 3 desenvolvedores são júnior;
- Gravação ou documentação escrita do processo de publicação nas lojas de aplicativos, por ser uma etapa pouco frequente e fácil de esquecer entre uma release e outra.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

O projeto não prevê subcontratados; as dependências de terceiros relevantes são as bibliotecas de código aberto usadas na web e no mobile, tratadas como itens de configuração e versionadas via lockfile (package-lock.json/yarn.lock, Podfile.lock, etc.).

- Dependabot habilitado no repositório para análise automática de vulnerabilidades e abertura de PRs de atualização;
- Atualizações de dependências seguem o mesmo fluxo de PR + revisão + pipeline das demais mudanças;
- Serviços de terceiros usados em produção (ex.: provedor de e-mail transacional, gateway de doação/pagamento) devem ter suas versões de API e credenciais documentadas em /docs, com as credenciais mantidas apenas no GitHub Secrets.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção, contemplando as trilhas web e mobile:

Etapa	Descrição	Gatilho
CI	Build, lint, testes unitários/integração, análise de dependências, geração de artefato/pacote de teste.	Abertura/atualização de Pull Request
Merge	Integração da mudança na branch main.	1 aprovação + CI verde
CD — Staging (Web)	Deploy automático em homologação (PaaS).	Merge na branch main

Etapa	Descrição	Gatilho
CD — Teste interno (Mobile)	Publicação automática em Internal Testing / TestFlight.	Merge na branch main
Gate de Release	Aprovação manual do Desenvolvedor Líder para produção/lojas.	Sucesso da validação em staging/teste interno
CD — Produção (Web)	Deploy via PaaS, health check, geração de tag.	Aprovação do gate de release
CD — Lojas (Mobile)	Submissão à revisão da Google Play/Apple, lançamento faseado quando disponível.	Aprovação do gate de release